

Code Repair using Recurrent Neural Networks and Transformer Architectures

Tilak Asodariya

June 3, 2026

Abstract

This report presents a comparative study of sequence-to-sequence architectures for automated code repair. Multiple recurrent neural network models and transformer-based models were implemented and evaluated on a code repair dataset. The study investigates the effect of bidirectional encoding, dropout regularization, attention mechanisms, and transformer architectures on repair performance. In addition to using the PyTorch Transformer implementation, a custom Transformer was developed from scratch, including self-attention, multi-head attention, positional encoding, masking mechanisms, encoder-decoder attention, and feed-forward networks.

1 Introduction

Automated code repair is the task of transforming buggy source code into its corrected version. This problem can be formulated as a sequence-to-sequence learning task where the input sequence corresponds to buggy code and the output sequence corresponds to repaired code.

The objectives of this work were:

- Implement baseline recurrent sequence-to-sequence models.
- Study the effect of bidirectional encoding.
- Analyze the impact of dropout regularization.
- Evaluate attention mechanisms.
- Implement transformer architectures.
- Develop a custom Transformer from scratch.
- Compare the performance of recurrent and transformer-based approaches.

2 Dataset and Preprocessing

The dataset consists of pairs of source code snippets:

- Buggy code
- Corresponding repaired code

The preprocessing pipeline involved:

1. Tokenizing code using whitespace-based tokenization.

2. Constructing a vocabulary from the training set.
3. Adding special tokens:
 - <PAD>
 - <SOS>
 - <EOS>
 - <UNK>
4. Encoding tokens into integer indices.
5. Padding sequences for batch processing.

For rapid experimentation, a subset containing:

- 10,000 training samples
- 2,000 validation samples

was initially used. Later experiments with transformers were also performed on the complete dataset.

3 Evaluation Metric

3.1 Token Accuracy

Token accuracy was used as the primary evaluation metric.

$$\text{Token Accuracy} = \frac{\text{Correct Tokens}}{\text{Total Tokens}} \times 100 \quad (1)$$

This metric measures the percentage of generated tokens that exactly match the target sequence.

4 Baseline Models

4.1 LSTM Encoder-Decoder

The first model consisted of:

- LSTM Encoder
- LSTM Decoder
- Teacher Forcing during training

The decoder relied solely on the final hidden state produced by the encoder.

4.2 BiLSTM Encoder-Decoder

The encoder was replaced with a Bidirectional LSTM.

Advantages:

- Access to past context.
- Access to future context.
- Improved modeling of source-code dependencies.

4.3 BiLSTM with Dropout

Dropout regularization was introduced to investigate whether reducing overfitting would improve performance.

4.4 BiLSTM with Attention

An attention mechanism was added to the BiLSTM architecture.

Instead of relying only on the final encoder state, the decoder dynamically attended to encoder outputs during every decoding step.

Benefits include:

- Better handling of long sequences.
- Improved token alignment.
- Direct access to source-code information.

5 Transformer Architectures

5.1 PyTorch Transformer

A standard encoder-decoder Transformer architecture was implemented using PyTorch's built-in Transformer module.

Major components included:

- Embedding layer
- Positional encoding
- Transformer encoder
- Transformer decoder
- Causal masking
- Output projection layer

5.2 Custom Transformer

To understand transformer internals, a complete Transformer was implemented manually.

The following components were developed from scratch:

- Self-Attention
- Multi-Head Attention
- Feed Forward Networks
- Positional Encoding
- Encoder Layers
- Decoder Layers
- Encoder-Decoder Attention
- Residual Connections
- Layer Normalization
- Masking Mechanisms

5.2.1 Self-Attention

For a given query, key, and value matrix:

$$Attention(Q, K, V) = Softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

Self-attention enables each token to gather information from other tokens in the sequence.

5.2.2 Multi-Head Attention

Each attention head independently computes:

$$head_i = Attention(Q_i, K_i, V_i) \quad (3)$$

The outputs are concatenated and projected:

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O \quad (4)$$

This allows the model to learn multiple relationships simultaneously.

5.2.3 Encoder-Decoder Attention

The decoder uses:

- Query = Decoder representation
- Key = Encoder memory
- Value = Encoder memory

This enables the decoder to retrieve relevant information from the source sequence while generating repaired code.

6 Experimental Results

6.1 Baseline Models

Experiments were conducted using:

- 10,000 training samples
- 2,000 validation samples
- 5 training epochs

Model	Train Loss	Validation Loss	Token Accuracy (%)
LSTM	1.6580	2.8098	31.88
BiLSTM	1.4797	2.5878	35.21
BiLSTM + Dropout	1.5443	2.6170	34.71
BiLSTM + Attention	0.9509	2.1836	52.10

Table 1: Performance of recurrent architectures

6.2 Transformer Models on Reduced Dataset

Model	Token Accuracy (%)
PyTorch Transformer	50.92
Custom Transformer	48.60

Table 2: Transformer performance on the reduced dataset

6.3 Transformer on Full Dataset

Model	Token Accuracy (%)
PyTorch Transformer (Full Dataset)	61.64

Table 3: Transformer performance on the full dataset

7 Observations

7.1 BiLSTM vs LSTM

Using bidirectional encoding improved performance:

- LSTM: 31.88%
- BiLSTM: 35.21%

This demonstrates the importance of incorporating both past and future context.

7.2 Effect of Dropout

Dropout produced only a minor change in performance:

- BiLSTM: 35.21%
- BiLSTM + Dropout: 34.71%

This suggests that the baseline model was not severely overfitting.

7.3 Effect of Attention

Attention produced the largest improvement among recurrent architectures:

- BiLSTM: 35.21%
- BiLSTM + Attention: 52.10%

The decoder benefited from direct access to encoder outputs rather than relying solely on the final hidden state.

7.4 Transformer Performance

The Transformer achieved competitive performance even on the reduced dataset.

When trained on the full dataset, performance improved significantly:

- Reduced Dataset: 50.92%
- Full Dataset: 61.64%

This demonstrates the ability of transformer architectures to leverage larger datasets effectively.

7.5 Custom Transformer Validation

The custom Transformer achieved 48.60% token accuracy.

The relatively small gap compared to the PyTorch implementation indicates that the manually implemented:

- Self-Attention
- Multi-Head Attention
- Encoder-Decoder Attention
- Feed Forward Networks
- Masking Mechanisms

were functioning correctly.

8 Qualitative Analysis

The attention-based and transformer models generally:

- Preserved most of the original source code.
- Modified localized buggy regions.
- Maintained syntactic structure.

Transformer-based models demonstrated stronger handling of long-range dependencies and produced more consistent repairs than basic recurrent architectures.

9 Conclusion

This work investigated multiple sequence-to-sequence architectures for automated code repair.

The experiments show that:

1. Bidirectional encoding improves source-code understanding.
2. Dropout provides limited benefit in this setting.
3. Attention mechanisms significantly improve repair quality.
4. Transformer architectures scale effectively with larger datasets.

5. A custom Transformer can achieve performance close to optimized framework implementations.

Among recurrent architectures, the BiLSTM with Attention achieved the best performance. Overall, the Transformer trained on the complete dataset achieved the highest token accuracy of 61.64%, making it the strongest model evaluated in this study.